FLE TRIBE

WEEK 15 · WEDNESDAY · LESSON GUIDE

Search UX: debouncing & race conditions

This week: Load, cache and display real data in React, with loading and error states.

Debouncing

Wait until the user pauses typing before searching:

```
function useDebounce(value, delay = 400) {
  const [debounced, setDebounced] = useState(value);
  useEffect(() => {
    const id = setTimeout(() => setDebounced(value), delay);
    return () => clearTimeout(id);
  }, [value, delay]);
  return debounced;
}
```

Race conditions

If you search "re" then "react", the "re" results might arrive last and overwrite the right ones. Cancel the old request when a new one starts:

```
useEffect(() => {
  const controller = new AbortController();
  fetch(url(query), { signal: controller.signal })
    .then((r) => r.json())
    .then(setResults)
    .catch(() => {});
  return () => controller.abort();
}, [query]);
```

Keep the search in the URL

Store the query in search params (`?q=react`) so people can share and refresh a search.

Research questions

Answer these in the app, in your own words, to unlock the next lesson.

1. Type fast in a search box. How many requests should fire?
2. Write a `useDebounce` hook.

Start your research here

- MDN: https://developer.mozilla.org/en-US/docs/Glossary/Debounce
- MDN: https://developer.mozilla.org/en-US/docs/Web/API/AbortController